

Mathematical Formulation of the Dispatch Optimization Algorithm

bess-stack-de
 Degradation-aware rolling-horizon dispatch
 for a grid-connected BESS in the German market

V1.0

1 Scope and Conventions

This document states the optimization model implemented in `src/bess_stack/model/dispatch.py`. The project models

a degradation-aware, rolling-horizon dispatch optimizer for a single grid-connected BESS in the German market, co-optimizing day-ahead arbitrage, FCR capacity, and aFRR capacity + energy, that outputs an annual revenue stack and feeds a project-level IRR calculation.

Section 2 describes what is implemented. Section 3 describes what is not. Every equation in Section 2 corresponds to a statement in `solve_window`; no equation there describes intended behaviour. Section 3 formulates the reserve products from the parameters already present in `scenarios/reference.yaml`, none of which the code reads today. The distinction is maintained deliberately: a formulation that does not say which half is running is not a specification but an advertisement.

1.1 Units and sign conventions

Power is in MW, energy in MWh, prices and costs in EUR/MWh, and capacity prices in EUR/MW/h. One time step spans Δt hours. Charging and discharging power are separate non-negative variables rather than one signed variable, which is what allows the complementarity condition of constraint group B1 to be written as a linear constraint. State of charge is carried as an energy E_t in MWh rather than a percentage; the scenario file states its bounds as fractions and the model multiplies them by the nominal capacity on load.

2 Day-Ahead Arbitrage Model (Implemented)

2.1 Index sets

Symbol	Description
$t \in \mathcal{T}$	Time steps within one optimization window, $\mathcal{T} = \{0, 1, \dots, N-1\}$
$\mathcal{B}_j$	Steps belonging to day-ahead product block j , so that $\bigcup_j \mathcal{B}_j = \mathcal{T}$

2.2 Parameters

Each parameter names the field it is read from, so that the document and the configuration cannot drift apart unnoticed. Source fields are dotted paths into `scenarios/reference.yaml`; those beginning with a stream name (`day_ahead`, `fcr`, `afrr`) are relative to `revenue_streams`.

Symbol	Description	Unit	Source field
Δt	Length of one time step	h	<code>market.resolution_minutes</code>
π_t	Day-ahead price in step t	EUR/MWh	price series argument
κ	Spread retention, $1 - \text{haircut}$	–	<code>day_ahead.spread_haircut</code>
f	Trading fee per MWh traded	EUR/MWh	<code>day_ahead.fees_eur_per_mwh</code>
g^c	Network fee on energy charged	EUR/MWh	<code>grid.charging_fees_eur_per_mwh</code>
g^d	Network fee on energy discharged	EUR/MWh	<code>grid.discharging_fees_eur_per_mwh</code>
c^{deg}	Marginal degradation cost	EUR/MWh	<code>degradation.marginal_cost_eur_per_mwh</code>
$\bar{P}$	Binding power cap	MW	<code>min(battery.power_mw, grid.connection_limit_mw)</code>
E^{nom}	Nominal usable energy	MWh	<code>battery.energy_mwh</code>
η^c	One-way charging efficiency	–	<code>battery.efficiency.charge</code>
η^d	One-way discharging efficiency	–	<code>battery.efficiency.discharge</code>
σ	Self-discharge, fraction of E^{nom} per day	1/d	<code>battery.efficiency.self_discharge_per_day</code>
P^{aux}	Parasitic auxiliary load	MW	<code>battery.aux_load_mw</code>
L	Idle energy loss per step, see (2.5)	MWh	derived
E^{min}	Lower energy bound	MWh	<code>battery.soc.min</code> $\times E^{\text{nom}}$
E^{max}	Upper energy bound	MWh	<code>battery.soc.max</code> $\times E^{\text{nom}}$
E_{init}	Energy at window start	MWh	carried between windows
E^{term}	Required energy at window end	MWh	<code>dispatch.terminal_soc_fraction</code> $\times E^{\text{nom}}$
m	Time steps per day-ahead block	–	<code>day_ahead.product_resolution_minutes</code>
H	Optimization horizon	h	<code>dispatch.horizon_hours</code>
C	Committed step of the rolling horizon	h	<code>dispatch.commit_step_hours</code>

2.3 Decision variables

Variable	Domain	Unit	Description
P_t^c	$[0, \bar{P}]$	MW	Charging power in step t
P_t^d	$[0, \bar{P}]$	MW	Discharging power in step t
u_t	$\{0, 1\}$	–	Mode indicator, $1 = \text{charging permitted}$, $0 = \text{discharging permitted}$
E_t	$[E^{\text{min}}, E^{\text{max}}]$	MWh	Stored energy at the end of step t

2.4 Objective function

The model maximizes net trading revenue over the window:

$$\max \sum_{t \in \mathcal{T}} \left[(\pi_t \kappa - f - g^d - c^{\text{deg}}) P_t^d \Delta t - (\pi_t \kappa + f + g^c) P_t^c \Delta t \right] \quad (2.1)$$

Equation (2.1) is the sum over all steps of revenue from selling less the cost of buying. Its terms are:

- $\pi_t \kappa$ — the price actually captured. The haircut κ shaves the traded spread to represent the gap between the price used in the model and the price a real bid achieves.

- f, g^c, g^d — per-MWh fees. Note the asymmetry of sign: fees subtract from the sale price and *add* to the purchase price, so they narrow the spread from both ends rather than acting as a levy on net position.
- c^{deg} — the marginal cost of degradation, charged per MWh discharged. This is not a cash cost and nobody invoices it. It is the shadow price of consuming battery life, and it is what makes the model degradation-aware rather than merely degradation-reporting: a cycle whose spread does not cover c^{deg} is declined. Its admissible range is bounded above by the cost of making good the capacity one cycle consumes,

$$c^{\text{deg}} \leq \phi \cdot \kappa^{\text{aug}} \cdot 1000,$$

where ϕ is the cyclic fade per full cycle and κ^{aug} the augmentation cost per kWh, both read from the `degradation` block. Nominal energy cancels out of the ratio, because fade per cycle is a fraction of beginning-of-life capacity and one equivalent full cycle discharges exactly that. `config.py` enforces the bound. There is no corresponding lower bound: establishing one needs a discount factor keyed to the augmentation date, which depends on the cycle count, which is an outcome of dispatch rather than an input to it. Satisfying the bound therefore shows the value is not indefensibly high, not that it is right.

- Δt — converts power in MW to energy in MWh over the step.

Because degradation is charged on discharge only, throughput is measured as discharged energy throughout this document.

2.5 Operational constraints

B1 — Charge/discharge complementarity and power limits

A battery may not charge and discharge in the same step. The restriction is enforced through the binary mode indicator, which simultaneously imposes the power limit:

$$P_t^c \leq \bar{P} u_t \quad \forall t \in \mathcal{T} \quad (2.2)$$

$$P_t^d \leq \bar{P} (1 - u_t) \quad \forall t \in \mathcal{T} \quad (2.3)$$

where $\bar{P}$ is the smaller of the battery power rating and the grid connection limit, and u_t selects the mode.

This binary is not a modelling nicety and must not be relaxed to obtain a linear program. Under the relaxation, at sufficiently negative prices the optimizer charges and discharges simultaneously: the two positions cancel in the objective at zero net cost, while round-trip losses destroy stored energy. Destroying energy for free is valuable when prices are negative, because it lets a full battery keep buying. The behaviour is an artefact of the relaxation rather than a dispatch strategy, and the binary is what forbids it.

B2 — State-of-charge dynamics

Stored energy evolves under charging, discharging and idle losses:

$$E_t = E_{t-1} + \eta^c P_t^c \Delta t - \frac{P_t^d \Delta t}{\eta^d} - L \quad \forall t \in \mathcal{T} \quad (2.4)$$

with $E_{-1} = E_{\text{init}}$, and the idle loss per step

$$L = \left(\frac{\sigma E^{\text{nom}}}{24} + P^{\text{aux}} \right) \Delta t. \quad (2.5)$$

where η^c *multiplies* the charging term and η^d *divides* the discharging term. The asymmetry is deliberate and is the correct treatment of one-way efficiencies: of the energy drawn from the grid only a fraction η^c is stored, whereas delivering P_t^d to the grid requires drawing P_t^d/η^d from storage. The product $\eta^c\eta^d$ is the round-trip efficiency. The loss term L is independent of dispatch and applies in every step, including idle ones.

B3 — Energy bounds

$$E^{\min} \leq E_t \leq E^{\max} \quad \forall t \in \mathcal{T} \quad (2.6)$$

where the bounds are the scenario's state-of-charge fractions scaled by nominal capacity. In the implementation these are variable bounds rather than explicit rows.

B4 — Product block coherence

Day-ahead energy settles in hourly blocks, so dispatch may not vary within a block even when the model runs at a finer resolution:

$$P_t^c = P_{t-1}^c \quad \forall t : t \bmod m \neq 0 \quad (2.7)$$

$$P_t^d = P_{t-1}^d \quad \forall t : t \bmod m \neq 0 \quad (2.8)$$

where m is the number of model steps in one product block. With a 15-minute resolution and a 60-minute day-ahead product, $m = 4$. This is also the machinery the four-hour reserve blocks of Section 3 will reuse, which is why it is expressed generally rather than hard-coded to the hourly case.

B5 — Terminal state of charge

$$E_{N-1} \geq E^{\text{term}} \quad (2.9)$$

where E^{term} is the required closing energy. The constraint exists because stored energy has no value beyond the horizon: without it the optimizer empties the battery into the final step of every window, since selling is free revenue and the consequence falls outside the model. It is applied at the far end of the window, not at the end of the committed region, so that it shapes end effects rather than the decisions actually taken.

2.6 Rolling-horizon procedure

The window model above is solved repeatedly across the price series. With $h = H/\Delta t$ steps of horizon and $c = C/\Delta t$ steps of commitment:

1. Set E_{init} to the opening state of charge.
2. Solve (2.1)–(2.9) over the next h steps.
3. Retain the first c steps of the solution as committed dispatch.
4. Set E_{init} to E_{c-1} from the solved window.
5. Advance by c steps and repeat until the series is exhausted.

The horizon exceeds the commitment ($h > c$) so that the uncommitted tail absorbs the distortion of B5. A final window shorter than one commitment step receives no terminal target, since there is no tail left to absorb it.

Setting `dispatch.mode` to `perfect_foresight` collapses the procedure to a single window spanning the entire series. This reports an upper bound on achievable revenue rather than an expectation, and the same is true of the rolling mode while `dispatch.forecast.kind` remains `perfect`: each window sees its own prices exactly.

2.7 Post-optimization network charges

Network charges that fall outside the exemption of §118(6) EnWG are applied in `src/bess_stack/finance/grid_fees.py` to an already-solved dispatch, rather than by re-solving each project year under that year's tariff. For a year y the charge is

$$\Phi_y = (\gamma_y - \gamma^{\text{priced}}) \sum_t P_t^c \Delta t + \mathbb{I}[y \text{ not exempt}] \cdot \rho \cdot \bar{P}^{\text{conn}} \cdot 1000 \quad (2.10)$$

where γ_y is the per-MWh charging fee applicable in year y , γ^{priced} is the fee the optimizer already carried in (2.1) via g^c , ρ is the successor capacity charge in EUR/kW/year and $\bar{P}^{\text{conn}}$ is the connection capacity in MW. Subtracting the fee already priced is what prevents a charge from being billed twice.

This is an approximation and its error is measured rather than assumed. Because the retained dispatch was optimized without the fee, it remains feasible but no longer optimal once the fee exists, so *revenue is understated*; and because the marginal cycles the fee should have suppressed are still present, *throughput and cycle count are overstated*. Measured against a re-solved run on a full synthetic year, a 5 EUR/MWh charging fee understates revenue by 1.0% while overstating throughput by 13.3%; at 25 EUR/MWh the figures are 50.7% and 101.1%. The asymmetry matters, because the augmentation trigger keys off cycle count and therefore fires early long before the revenue figure looks wrong. The error is exactly zero while the exemption holds.

3 Reserve Products (Specified, Not Implemented)

Nothing in this section is implemented. The parameters below are present in `scenarios/reference.yaml`, but `config.py` defines no FCR or aFRR dataclass and does not read them, and `solve_window` builds no reserve variables. What follows is the formulation these parameters imply, recorded so the schema and the intended model can be checked against each other before either is built.

3.1 Additional sets and parameters

Let $k \in \mathcal{K}$ index four-hour reserve blocks, with $\mathcal{T}_k$ the model steps in block k and H^{blk} the block length in hours.

Symbol	Description	Source field
c^{FCR}	FCR capacity price (EUR/MW/h)	<code>fcr.capacity_price_eur_per_mw_h</code>
h^{FCR}	Energy headroom held per MW awarded (h)	<code>fcr.reserve_energy_hours</code>
R^{min}	Minimum bid size (MW)	<code>fcr.min_bid_mw, afrr.min_bid_mw</code>
$c^{a\pm}$	aFRR capacity price, positive direction shown; negative is the counterpart	<code>afrr.positive.capacity_price_eur_per_mw_h</code>
$\pi^{a\pm}$	aFRR energy price, likewise per direction	<code>afrr.positive.energy_price_eur_per_mwh</code>
$\alpha^\pm$	Expected activation rate, per direction	<code>afrr.positive.activation_rate</code>
h^a	aFRR energy headroom per MW awarded (h)	<code>afrr.reserve_energy_hours</code>

3.2 Additional variables

$R_k^{\text{FCR}} \geq 0$ is the symmetric FCR award in block k ; $R_k^{a+}, R_k^{a-} \geq 0$ are the directional aFRR awards. Binary indicators $y_k^\bullet \in \{0, 1\}$ carry the minimum bid size.

3.3 Revenue terms

The objective (2.1) gains capacity revenue and expected activation revenue:

$$+ \sum_{k \in \mathcal{K}} H^{\text{blk}} \left[c^{\text{FCR}} R_k^{\text{FCR}} + c^{a+} R_k^{a+} + c^{a-} R_k^{a-} + \alpha^+ \pi^{a+} R_k^{a+} - \alpha^- \pi^{a-} R_k^{a-} \right] \quad (3.1)$$

where the sign on the negative-direction energy term reflects that π^{a-} is quoted as payment *to* the provider for absorbing energy, so a negative price is revenue. This convention must be pinned down against settlement rules before implementation; it is stated here precisely because it is the kind of sign that is easy to get backwards and hard to detect afterwards.

3.4 R1 — Minimum bid size

$$R^{\min} y_k^\bullet \leq R_k^\bullet \leq \bar{P} y_k^\bullet \quad \forall k \in \mathcal{K} \quad (3.2)$$

where $\bullet$ ranges over the three products. An award is either zero or at least $R^{\min}$, which is a semi-continuous variable and the reason reserve participation introduces binaries of its own.

3.5 R2 — Power headroom

Capacity sold must be deliverable on top of energy dispatch:

$$P_t^d + R_k^{\text{FCR}} + R_k^{a+} \leq \bar{P} \quad \forall k, \forall t \in \mathcal{T}_k \quad (3.3)$$

$$P_t^c + R_k^{\text{FCR}} + R_k^{a-} \leq \bar{P} \quad \forall k, \forall t \in \mathcal{T}_k \quad (3.4)$$

3.6 R3 — Energy headroom

Capacity must also be sustainable for the prequalification duration, which is what h^{FCR} and h^a encode:

$$E_t - (h^{\text{FCR}} R_k^{\text{FCR}} + h^a R_k^{a+}) \geq E^{\min} \quad \forall k, \forall t \in \mathcal{T}_k \quad (3.5)$$

$$E_t + (h^{\text{FCR}} R_k^{\text{FCR}} + h^a R_k^{a-}) \leq E^{\max} \quad \forall k, \forall t \in \mathcal{T}_k \quad (3.6)$$

where $h^{\text{FCR}} = 0.5$ reflects the German requirement that full FCR activation be sustainable for 30 minutes in both directions. Understating it permits the same MW to be sold to reserve and to arbitrage at once.

3.7 Known gap: activation does not move the state of charge

In the formulation above, aFRR energy enters revenue through the expected activation rate $\alpha^\pm$ but does not appear in the state-of-charge dynamics (2.4). Real activation moves energy and therefore constrains subsequent dispatch. Closing that gap requires either activation as a deterministic energy flow inside B2, or a stochastic formulation over activation scenarios. Until it is closed, reserve participation will be overvalued, because the model collects activation revenue without paying its state-of-charge consequence.

4 Execution and Reported Quantities

4.1 Running a scenario

One scenario file in, one revenue summary out:

```
python scripts/run_scenario.py scenarios/reference.yaml # full year
python scripts/run_scenario.py scenarios/reference.yaml --days 7 # first 7 days
```

`bess-stack-run` is the same code installed as a console script, and appears once the package has been reinstalled, since the installer is what generates it.

4.2 Definition of the reported quantities

The summary reports six quantities, defined here in terms of the decision variables of Section 2.3 so that a number in the output can be traced to the model that produced it.

$$\text{gross revenue} = \sum_{t \in \mathcal{T}} \pi_t (P_t^d - P_t^c) \Delta t \quad (4.1)$$

$$\text{energy discharged} = \sum_{t \in \mathcal{T}} P_t^d \Delta t \quad (4.2)$$

$$\text{energy charged} = \sum_{t \in \mathcal{T}} P_t^c \Delta t \quad (4.3)$$

$$\text{degradation cost} = c^{\text{deg}} \sum_{t \in \mathcal{T}} P_t^d \Delta t \quad (4.4)$$

$$\text{net revenue} = \text{gross revenue} - \text{degradation cost} \quad (4.5)$$

$$\text{equivalent full cycles} = \frac{1}{E_{\text{nom}}} \sum_{t \in \mathcal{T}} P_t^d \Delta t \quad (4.6)$$

Note that (4.1) is evaluated at the raw price π_t , whereas the objective (2.1) is evaluated at the captured price $\pi_t \kappa$ net of fees. The reported net revenue is therefore *not* the objective value in general: the two differ by

$$(1 - \kappa) \sum_t \pi_t (P_t^d - P_t^c) \Delta t + (f + g^d) \sum_t P_t^d \Delta t + (f + g^c) \sum_t P_t^c \Delta t. \quad (4.7)$$

In the reference scenario $\kappa = 1$ and $f = g^c = g^d = 0$, so the difference vanishes and the two coincide. That is a property of the reference case rather than an identity, and it stops holding the moment a haircut or a fee is set.

Energy discharged is the quantity degradation is charged on, and equivalent full cycles is that same energy expressed in units of nominal capacity. The two carry the same information and are reported together only because the cycle count is the figure that feeds the augmentation trigger.

4.3 Reference case

Measured on the reference scenario over a full synthetic year, not illustrative. The summary rounds for display; the exact values are given alongside because Section 5 checks arithmetic against them.

Quantity	As printed	Exact
Gross revenue	473,374 EUR	473,373.8047
Degradation cost	46,104 EUR	46,103.5848
Net revenue	427,270 EUR	427,270.2200
Energy discharged	11,526 MWh	11,525.8962
Energy charged	13,565 MWh	13,565.0702
Equivalent full cycles	576.3	576.2948

The gap between energy charged and energy discharged is round-trip loss plus the idle loss L of (2.5), and is the physical content of the efficiency asymmetry described under constraint

group B2. Section 5 traces these figures back to the scenario file and states the identities that check them.

4.4 What the reported figures exclude

The summary carries five caveats. They are not decoration on the output; they are the difference between what the scenario describes and what the model computes, and each is a restatement in operational terms of a boundary already drawn in this document.

- **Day-ahead only.** `revenue_streams.fcr` and `revenue_streams.afrr` are enabled in the reference scenario, but Section 3 is not implemented, so every figure is the day-ahead leg alone and understates the stack the scenario describes.
- **Returns repeat one modelled year.** `finance/cashflow.py` computes the quantities `outputs.metrics` names, but from a single solved year whose margin is scaled by remaining capacity rather than from a year-by-year re-solve. The scaling is linear in usable energy, which is conservative, because the cycles a smaller battery declines are its least valuable ones.
- **The reference case outlives its own battery.** Capacity crosses `degradation.end_of_life_capacity_fraction` in 2041 with both permitted augmentation events spent, so the closing years book revenue from a pack past end of life. The summary reports the year; the metrics are not corrected for it.
- **Synthetic prices by default.** `market.price_source` is `synthetic`, so the figures measure the price generator of `data/prices.py` rather than the German market. `smard` fetches real DE-LU day-ahead prices through `data/smard.py`; `entsoe` remains unimplemented and raises rather than silently degrading.
- **Short runs are not annualised.** `--days N` models the first N days of the series, which begin in January. January carries the narrowest spreads in the synthetic year, so pro-rating a short run understates the year rather than approximating it. Seven days yield 5,940 EUR net, which pro-rates to roughly 310,000 EUR against the 427,270 EUR measured above — an understatement of about 27%.

Network charges are reported separately and are zero in every year of the reference case, for the reason given in Section 2.7: the exemption of §118(6) EnWG runs to 2044 and the modelled life is 2025–2044, so no project year is charged.

5 Worked Example: How the Reference Case Is Computed

This section traces the figures of Section 4.3 from the scenario file to the printed summary, and states the identities that check them. It exists so the numbers can be audited without running the model, and so that a change which breaks one of the identities is visibly a change rather than a new number.

5.1 Derived constants

Nothing below is a scenario field. Each is computed from one or more fields at load or at solve time, and each enters the model in that derived form.

Symbol	Derivation	Value
Δt	<code>market.resolution_minutes</code> expressed in hours	0.25 h
N	$8760/\Delta t$	35,040
$\bar{P}$	lesser of <code>battery.power_mw</code> and <code>grid.connection_limit_mw</code>	10 MW
$E^{\min}$	<code>battery.soc.min</code> $\times E^{\text{nom}}$	1.0 MWh
$E^{\max}$	<code>battery.soc.max</code> $\times E^{\text{nom}}$	19.0 MWh
E_{init}	<code>battery.soc.initial</code> $\times E^{\text{nom}}$	10.0 MWh
E^{term}	<code>dispatch.terminal_soc_fraction</code> $\times E^{\text{nom}}$	10.0 MWh
m	<code>day_ahead.product_resolution_minutes</code> over <code>market.resolution_minutes</code>	4 steps
h	<code>dispatch.horizon_hours</code> $/\Delta t$	144 steps
c	<code>dispatch.commit_step_hours</code> $/\Delta t$	16 steps
L	$(\sigma E^{\text{nom}}/24 + P^{\text{aux}})\Delta t$	0.01270833 MWh
κ	$1 - \text{day_ahead.spread_haircut}$	1

5.2 The price series

`synthetic_day_ahead_hourly` returns 8760 hourly prices, which `upsample` repeats four times each to give the 35,040 quarter-hourly values the model consumes. Day-ahead settles hourly, so the price is constant inside each hour by construction and constraint group B4 keeps power constant there too.

Two properties of that series matter when reading any figure derived from it. The generator takes a `year` argument that its body never references, so the series is identical for every modelled year, and the seed is fixed at 42 and is not exposed through the scenario file: the reference case rests on one fixed realisation rather than a draw. And the calendar is synthetic. 8760 hours is exactly 365 days with no leap handling, and the weekday index is the day number modulo seven, so weekends fall on a repeating cycle that is not aligned to the modelled year's real calendar.

5.3 Window bookkeeping

Windows start at every c -th step, giving $35,040/16 = 2190$ windows. Each solves $h = 144$ steps and commits the leading $c = 16$, and 2190×16 recovers exactly 35,040, so every step is committed once and none twice.

The terminal target of constraint B5 is applied only where a window is longer than the committed region. The last window begins at step 35,024 and is exactly 16 steps long, so it receives none, and the battery is free to end the year empty. It does: $E_{N-1} = 1.0$ MWh, the lower bound. That is the end effect B5 exists to contain, visible here because the final window is the one place it cannot act.

5.4 The objective in the reference case

The reference scenario sets $\kappa = 1$ and $f = g^c = g^d = 0$, so the captured and paid prices of (2.1) both collapse to the raw price, and the sell and buy coefficient vectors become the same array. The objective reduces to

$$\max \sum_{t \in \mathcal{T}} \left[(\pi_t - c^{\text{deg}}) P_t^d - \pi_t P_t^c \right] \Delta t, \quad (5.1)$$

with $c^{\text{deg}} = 4$ EUR/MWh the only term that distinguishes the two directions. A step priced at 108.75 EUR/MWh therefore costs 108.75 to charge and earns 104.75 to discharge. Setting c^{deg} to zero as well would reduce (5.1) exactly to the gross revenue of (4.1), which is the sense in which gross revenue is this objective with degradation switched off.

5.5 Identities that check the reported figures

Three relations must hold exactly. They are worth re-running after any change to the objective or to constraint group B2, because each fails loudly rather than drifting.

First, degradation cost is throughput priced at c^{deg} : $4.0 \times 11,525.8962 = 46,103.5848$.

Second, net revenue is gross less that cost: $473,373.8047 - 46,103.5848 = 427,270.2199$, printed as 427,270.22.

Third, and the strongest of the three, the year's energy balance closes:

$$E_{\text{init}} + \eta^c \sum_t P_t^c \Delta t - \frac{1}{\eta^d} \sum_t P_t^d \Delta t - NL = E_{N-1} \quad (5.2)$$

Numerically $10.0 + 12,724.0358 - 12,287.7358 - 445.3000 = 1.0000$, against an observed final state of charge of 1.0000 MWh, for a residual of zero to eight decimal places. This is a stronger check than the first two, because it ties the charging and discharging sums, both efficiencies, the idle loss and the initial and final states into a single equation: an error in any one of them breaks it.

Two further properties were confirmed on the same run. State of charge stayed within $[1.0, 19.0]$ MWh, touching both bounds, and no step carried simultaneous charging and discharging, which is what constraint group B1 exists to prevent.

5.6 What the cycle count counts

Equivalent full cycles divide discharged energy by nominal capacity, and both halves of that ratio are conventions rather than facts.

Basis for the numerator	Energy (MWh)	Cycles
Discharged, measured at the grid (<i>used</i>)	11,525.8962	576.2948
Discharged, measured at the cells	12,287.7358	614.3868
Charged, measured at the grid	13,565.0702	678.2535

The numerator is discharged rather than charged energy, which is consistent: degradation is charged per MWh discharged, so the cycle count and the degradation cost share a numerator.

The denominator is nominal capacity, not the 18 MWh the state-of-charge band actually permits. A full traverse of that band is therefore 0.90 equivalent full cycles, and the battery cannot perform one whole cycle in a single pass. This is consistent with how fade is defined — fade per cycle is a fraction of beginning-of-life capacity, and one equivalent full cycle discharges exactly that — but it does mean 576.29 cycles never occurred as 576 traversals. They are the sum of many partial ones, at an average of 1.58 per day.

The remaining choice is open. Throughput is measured at the grid, so delivering 11,525.8962 MWh required drawing $11,525.8962/\eta^d = 12,287.7358$ MWh from the cells. Cell degradation physically tracks what passes through the cells, which is 6.61% more than what is counted here. Whether that is correct depends on how `degradation.cyclic_fade_per_full_cycle` was calibrated: against energy delivered, in which case the model is consistent, or against cell throughput, in which case fade is understated by that margin and the augmentation trigger fires later than it should. The scenario file does not say which, and until it does this is a known uncertainty of about 6.6% in the cycle count and in everything keyed to it.